

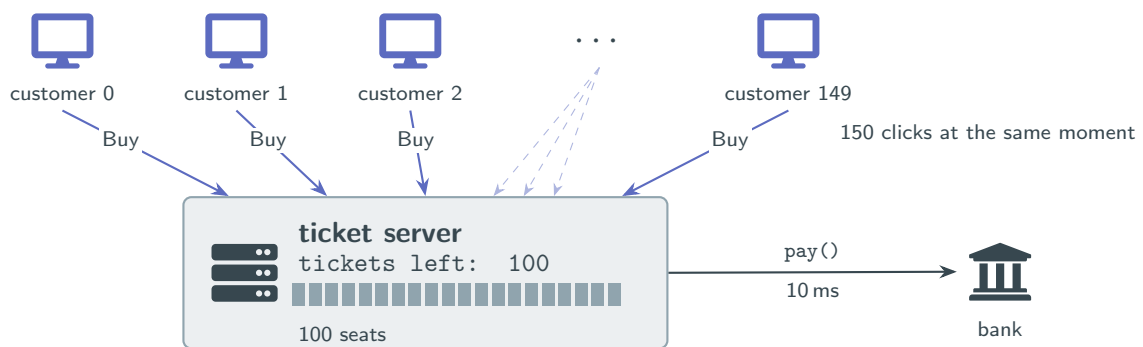
Computer Hardware ■ Lecture 8: Practice with Threads

Task 1: The ticket shop

A company has hired you to check the code of its ticket shop. Look at the code below and judge the dangers. Is there a race condition here? A deadlock? What do you think?

The situation

A concert hall has **100 seats**, and **150 people** want a ticket. Every customer is a separate thread with a unique number from 0 to 149, and all of them click *Buy* at the same time.



150 customers, 100 tickets: who gets one?

Ready code (in `tickets.h`, we do not look inside)

Functions

`pay()` simulates connecting to the bank and the customer paying for the ticket:
10 ms

Constants

`SEATS` 100 seats in the hall
`CUSTOMERS` 150 customers, one thread each

The code

The template is in [Task1](#). This is `tickets.cpp`:

```
int tickets = SEATS;           // tickets left
bool got[CUSTOMERS] = {false}; // did the customer get a ticket?

void customer(int id) {
    if (tickets > 0) {          // 1. check: any tickets left?
        pay();                 // (the customer pays)
        tickets--;             // 2. act: buy
        got[id] = true;
    }
}

int main() {
    std::vector<std::thread> threads;
    for (int id = 0; id < CUSTOMERS; id++)
        threads.emplace_back(customer, id);
    for (auto &t : threads)
        t.join();

    std::cout << "tickets left: " << tickets << "\n";
}
```

In the task folder, make `run` compile the program and runs it.

How `main()` starts 150 threads

In Lecture 7 every thread had its own variable: `std::thread a(lunch);`. For 150 threads we keep them in a list instead, a `std::vector`:

```
std::vector<std::thread> threads; // an empty list of threads
for (int id = 0; id < CUSTOMERS; id++)
    threads.emplace_back(customer, id); // start a new thread: customer(id)
for (auto &t : threads)                // for every thread in the list ...
    t.join();                          // ... wait until it ends
```

`threads.emplace_back(customer, id)` creates a new `std::thread` at the end of the list. The thread starts at once and runs `customer(id)`: the first one with `id = 0`, the last one with `id = 149`. It does the same as `std::thread t(customer, id);`, only the thread is kept in the vector, so that we can `join()` all of them later. The other tasks start their threads in the same way.

Your tasks

- 1 Do you see a danger? What exactly can go wrong? Suppose only one ticket is left, and two customers click *Buy* at the same moment. Describe, step by step, how both of them can get a ticket.
- 2 Write a function `check()` that checks whether the shop sold the tickets correctly: no more tickets than seats, and the tickets sold plus the tickets left add up to `SEATS`. It prints `OK` or `ERROR`. Call it at the end of `main()` and run the program several times. Does your check catch the problem every time?

- 3 Fix the danger. Does your check pass now?
- 4 `pay()` talks to the bank, and in real life it can take several seconds. While one customer pays under the mutex, all the others wait. It is possible to move `pay()` **outside** the mutex and keep the shop safe. Think how to do it, write it, and check it with `check()`. Measure how long the program takes before and after.
- 5 Would it be enough to change `int tickets` to `std::atomic<int> tickets` and keep the same `if`? Try it.